

Atelier Oracle Dev Week 2026

LangChain4j-CDI

IA générative nativement injectable dans Jakarta EE — Construisez des agents IA prêts pour la production avec CDI, MicroProfile et LangChain4j sur WildFly.



Yann Blazart
Tech Lead @ SCIAM



Emmanuel Hugonnet
WildFly Core @ IBM

Ce que vous allez construire

Dans cet atelier pratique, vous allez progressivement construire six applications alimentées par l'IA en utilisant **LangChain4j-CDI**, transformant des interfaces Jakarta EE en agents IA injectables avec une seule annotation.



Exercice 1 : Agent IA

Créez un skald viking injectable avec `@RegisterAIService`, des réponses en streaming et de la vision multimodale.



Exercice 2 : Résilience et observabilité

Ajoutez MicroProfile Fault Tolerance (`@Retry`, `@Timeout`, `@CircuitBreaker`) et OpenTelemetry.



Exercice 3 : Intégration MCP

Connectez votre agent IA à des outils externes via le Model Context Protocol (MCP) — « JDBC pour l'IA ».



Exercice 4 : Guardrails

Ajoutez des garde-fous sur l'entrée et la sortie de votre skald viking : langue française obligatoire et sans races fantastiques.



Exercice 5 : A2A Protocol

Orchestrez plusieurs agents IA via le protocole Agent-to-Agent : un writer, un scorer, et un orchestrateur avec boucle de révision déclarative.



Exercice 6 : Supervisor Agent

Remplacez la boucle explicite par un superviseur LLM avec `@RegisterSupervisorAgent` : le modèle décide lui-même quand la qualité est atteinte.



FORMAT DE L'ATELIER

Chaque exercice dispose d'un module `base/` (squelette avec TODOs) et d'un module `solution/` (référence complète). Travaillez dans `base/` et suivez les étapes. Si vous êtes bloqué, la `solution/` est toujours là comme filet de sécurité.

Prérequis

Logiciels requis

OUTIL	VERSION	UTILITÉ
JDK	21+	Développement Java
Maven	3.9+	Outil de build
IDE	Apache NetBeans / IntelliJ IDEA / VS Code	Éditeur de code
curl	n'importe quelle	Test d'API
Clé API Mistral AI ou Ollama	—	Fournisseur LLM — distant (Mistral AI, gratuit) ou local (Ollama)
Docker / Podman	n'importe quelle	Stack Grafana LGTM (Exercice 2 uniquement)

1. Récupérer le code source

```
git clone https://github.com/ehsavoie/langchain4j-cdi-lab-2026.git
cd langchain4j-cdi-lab-2026/demo-project
```

2. Choisir votre fournisseur LLM

 **DEUX OPTIONS AU CHOIX**

Option A — Mistral AI (distant) : inscrivez-vous gratuitement sur console.mistral.ai (<https://console.mistral.ai/>) pour obtenir une clé API. Aucune installation locale requise.

Option B — Ollama (local) : installez [Ollama](https://ollama.ai/) (<https://ollama.ai/>), puis téléchargez les modèles :

```
# Dans un premier terminal (laisser tourner) :
ollama serve

# Dans un second terminal :
ollama pull ministral-3:3b # exercices 1, 2, 3, 4
ollama pull qwen2.5:7b    # exercice 5 (A2A — toujours requis)
```

Ollama doit rester actif sur `http://localhost:11434` pendant tout l'atelier.

⚙️ COMMENT CONFIGURER LE FOURNISSEUR DANS CHAQUE EXERCICE

Chaque module contient un fichier `microprofile-config.properties` structuré ainsi :

```
# ---- Option A : Mistral AI (distant) ----
dev.langchain4j.cdi.plugin.my-model.class=... MistralAiChatModel
dev.langchain4j.cdi.plugin.my-model.config.api-key=${MISTRAL_API_KEY}
dev.langchain4j.cdi.plugin.my-model.config.model-name=mistral-small-latest

# ---- Option B : Ollama (local) ----
# dev.langchain4j.cdi.plugin.my-model.class=... OllamaChatModel
# dev.langchain4j.cdi.plugin.my-model.config.base-url=http://localhost:11434
# dev.langchain4j.cdi.plugin.my-model.config.model-name=ministral-3.3b

dev.langchain4j.cdi.plugin.my-model.config.temperature=0.9
dev.langchain4j.cdi.plugin.my-model.config.log-requests=true
```

Par défaut, l'Option A (Mistral AI) est active. Pour passer à Ollama :

1. **Commentez** les 3 lignes de l'Option A (ajoutez `#` devant)
2. **Décommentez** les 3 lignes de l'Option B (retirez les `#`)

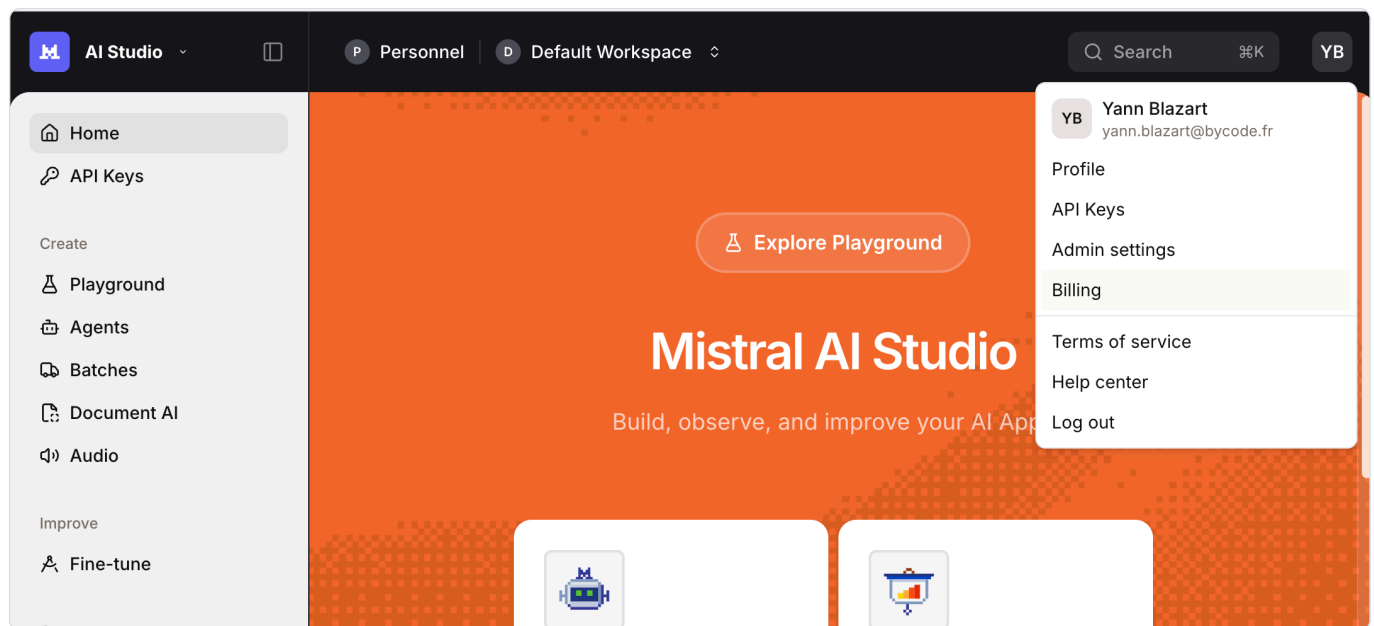
Les propriétés communes (`temperature`, `log-requests`, etc.) sont en dessous et **restent inchangées** — elles s'appliquent quel que soit le fournisseur choisi.

Option A : Configurer Mistral AI (distant)

Inscrivez-vous sur console.mistral.ai (<https://console.mistral.ai/>) et suivez ces étapes pour obtenir une clé API gratuite :

Étape 1 : Accéder aux clés API

Depuis la page d'accueil de Mistral AI Studio, cliquez sur l'icône de votre profil (coin supérieur droit), puis sélectionnez **API Keys**.



Étape 2 : Choisir un forfait

Sur la page API Keys, vous verrez un avertissement : *"No plan is currently active on this organization, the API keys will not be active."* Cliquez sur **Choose a plan** pour activer vos clés.

⚙️ Admin ▾

👤 Personnel ▾

YB

■ ADMINISTRATION

Organization

Access

Members

■ SUBSCRIPTIONS

Subscriptions

Billing

■ MANAGE

Le Chat

API Keys

Usage

Limits

Workspaces

Privacy

Organization API keys

As an admin of this organization, you can view and manage all API keys in this organization.

Create new key

No plan is currently active on this organization, the API keys will not be active. Choose a plan →

Active | Expired

📄 5

🔍 Search

Name ▾	Key ▾	Created ▾	Expires ▾	Workspace ▾
--------	-------	-----------	-----------	-------------

⚠️ IMPORTANT

Sans forfait actif, vos clés API seront créées mais **ne fonctionneront pas**. Vous obtiendrez une erreur `Unauthorized` lors de l'appel de l'API.

Étape 3 : Sélectionner le forfait gratuit "Experiment"

Choisissez le forfait **Experiment** (côté gauche) — il est complètement **gratuit** et vous donne accès à tous les modèles d'IA de pointe. Cliquez sur **"Experiment for free"**.

Choose plan

Unlock the full potential of AI Studio.

Experiment

Access without paying. API requests may be used to improve our services.

- ✓ Access frontier AI models
- ✓ Create and deploy agents
- ✓ Start experimenting quickly

Free

Experiment for free

Scale

Pay for what you use. Billed monthly or drawn from your credits.

- ✓ Pay-per-use; see pricing
- ✓ Access to frontier models
- ✓ Access to creation and deployment of agents
- ✓ Access to the fine-tuning API
- ✓ Max. 6 requests per second, scalable on demand
- ✓ Max. 2M input tokens per minute, scalable on demand
- ✓ Max. 10B input tokens per month, scalable on demand
- ✓ 20M tokens per minute, and 200M tokens per month for Mistral Embed
- ✓ Access to all features, including previews and beta features
- ✓ API requests are not used to train our models

Price **Pay-per-use** 

Scale as you go

Étape 4 : Accepter les conditions et s'abonner

Cochez la case pour accepter les conditions d'utilisation et la politique de confidentialité, puis cliquez sur **Subscribe**.

Sign up to Experiment

Access without paying.

Subscribe

By choosing the Experiment plan, you're agreeing that your anonymized requests may be used for research purposes and accept the Mistral AI [Terms of Service](#) and [Privacy Policy](#).

☐ I accept the Mistral AI [Terms of Service](#) and [Privacy Policy](#).

Cancel **Subscribe**

Experiment

Access without paying. API requests may be used to improve our services.

- ✓ Access frontier AI models
- ✓ Create and deploy agents
- ✓ Start experimenting quickly

Il vous sera ensuite demandé de fournir votre **numéro de téléphone** pour recevoir un **code d'activation par SMS**. Ceci est requis pour activer le forfait gratuit Experiment.

Étape 5 : Créer et exporter votre clé API

Une fois votre forfait activé, retournez sur la page API Keys et cliquez sur **"Create new key"**. Copiez la clé immédiatement — vous ne pourrez plus la revoir.

Puis exportez-la dans votre terminal :

```
# Linux / macOS
export MISTRAL_API_KEY=your-api-key-here

# Windows (PowerShell)
$env:MISTRAL_API_KEY="your-api-key-here"
```

💡 OPTION B : OLLAMA LOCAL

Si vous avez choisi Ollama, **ignorez toute la section Mistral AI ci-dessus**. Pas besoin de clé API — il suffit de commenter/décommenter les blocs dans `microprofile-config.properties` comme expliqué plus haut.

3. WildFly (automatique)

WildFly 39 est automatiquement téléchargé et provisionné par le plugin Maven. Aucune installation manuelle nécessaire :

```
# Compiler, provisionner WildFly et déployer l'application
cd demo-1-ai-agent/solution
mvn clean install

# Démarrer le serveur
./target/server/bin/standalone.sh      # Linux / macOS
target\server\bin\standalone.bat       # Windows
```

`mvn clean install` provisionne WildFly et déploie l'application dans `target/server/`. Lancez ensuite le serveur avec le script approprié à votre système.

3bis. Configurer le fournisseur LLM dans la solution

Avant de vérifier votre installation, assurez-vous que le fichier `demo-1-ai-agent/solution/src/main/resources/META-INF/microprofile-config.properties` est configuré pour votre fournisseur :

- **Option A — Mistral AI (distant)** : c'est la configuration par défaut. Vérifiez simplement que la variable `MISTRAL_API_KEY` est bien exportée dans votre terminal.
- **Option B — Ollama (local)** : pour **chaque bloc** (my-model, my-streaming-model, vision-model), commentez les 3 lignes de l'Option A et décommentez les 3 lignes de l'Option B.

```
# Exemple pour my-model — Option B : Ollama
# (commentez les 3 lignes Mistral AI ci-dessus, puis décommentez celles-ci)
dev.langchain4j.cdi.plugin.my-model.class=dev.langchain4j.model.ollama.OllamaChatModel
dev.langchain4j.cdi.plugin.my-model.config.base-url=http://localhost:11434
dev.langchain4j.cdi.plugin.my-model.config.model-name=minstral-3:3b
```

⚠ ATTENTION

Le fichier contient **3 modèles** (my-model, my-streaming-model, vision-model). Pensez à basculer **les trois** si vous utilisez Ollama.

4. Vérifier votre installation

```
# Compiler tous les modules pour télécharger les dépendances
cd demo-project
mvn clean install -DskipTests

# Test rapide de vérification
cd demo-1-ai-agent/solution
mvn clean install
./target/server/bin/standalone.sh      # Linux / macOS
target\server\bin\standalone.bat      # Windows

# Dans un autre terminal :
curl -X POST -H "Content-Type: text/plain" \
  -d "Sing me a heroic song" \
  http://localhost:8080/demo-1/api/chat
```

Vous pouvez aussi tester directement dans votre navigateur en ouvrant <http://localhost:8080/demo-1> (<http://localhost:8080/demo-1>) — l'interface web vous permet de discuter avec le skald viking.

✓ PRÊT !

Si vous obtenez une réponse du skald viking IA (via curl ou via le navigateur), votre environnement est correctement configuré. Arrêtez le serveur (Ctrl+C) et passez aux exercices.

Vue d'ensemble de l'architecture

LangChain4j-CDI est une extension CDI qui fait le pont entre **LangChain4j** (le framework IA Java) et les serveurs **Jakarta EE / MicroProfile**. Elle transforme des interfaces annotées en agents IA injectables.

Comment ça fonctionne

1. Scan

L'extension CDI scanne les interfaces `@RegisterAIService` au démarrage

2. Lire les attributs

Pour chaque interface : `chatModelName`, `tools`, `chatMemoryProviderName`, `contentRetrieverName`

3. Créer un bean synthétique

Un proxy LangChain4j est enveloppé dans un bean CDI géré

4. Résoudre les composants via SPI

Le SPI `LLMConfig` lit depuis MicroProfile Config : `dev.langchain4j.cdi.plugin.<name>.class=...`

5. Injecter !

`@Inject Assistant assistant;` — ça fonctionne tout simplement

L'avant / après

Sans CDI (verbeux)

```
ChatModel model =
    MistralAiChatModel.builder()
        .apiKey(System.getenv("MISTRAL_API_KEY"))
        .modelName("pixtral-large-latest")
        .temperature(0.3)
        .maxTokens(1000)
        .build();

ChatMemory memory =
    MessageWindowChatMemory.builder()
        .maxMessages(20).build();

Assistant assistant =
    AIServices.builder(Assistant.class)
        .chatLanguageModel(model)
        .chatMemory(memory)
        .tools(new VikingTools())
        .build();
```

Avec CDI (propre)

```
@RegisterAIService(
    chatModelName = "my-model",
    tools = VikingTools.class)
public interface Assistant {

    @SystemMessage("You are a
        Viking expedition guide")
    String chat(
        @UserMessage String message);
}

// In your REST endpoint:
@Inject
Assistant assistant;
```

LangChain4j-CDI vs Spring AI

Si vous venez du monde Spring, vous connaissez peut-être **Spring AI**. Les deux frameworks offrent des capacités similaires — voici comment ils se comparent en termes de style d'API :

✂ MÊME PUISSANCE. ADN DIFFÉRENT.

Spring AI est excellent dans son écosystème. LangChain4j-CDI fait le même travail avec l'idiome Jakarta EE : déclaratif, injectable, portable.

♣ Spring AI / Spring Boot 3.x

```
// Configuration bean
@Configuration
public class AssistantConfig {
    @Bean
    ChatClient chatClient(
        ChatClient.Builder b,
        VectorStore vs,
        ChatMemory mem) {
        return b
            .defaultSystem("You are helpful")
            .defaultAdvisors(
                new QuestionAnswerAdvisor(vs),
                new MessageChatMemoryAdvisor(mem))
            .defaultTools(new WeatherTools())
            .build();
    }
}

// Service that uses it
@Service
public class AssistantService {
    private final ChatClient client;

    String chat(String user, String msg) {
        return client.prompt().user(msg)
            .advisors(a -> a.param(CONV_ID, user))
            .call().content();
    }
}
```

~25 lignes · 2 classes · 1 builder chain

♦ LangChain4j-CDI / Jakarta EE

```
// That's it. One interface.
@RegisterAIService(
    chatMemoryProviderSupplier
        = MemoryProvider.class,
    tools = WeatherTools.class,
    retrievalAugmentor = RagAugmentor.class)
public interface Assistant {

    @SystemMessage("You are helpful")
    String chat(
        @MemoryId String user,
        @UserMessage String msg);
}

// Inject anywhere. Done.
@Inject
Assistant assistant;
assistant.chat("yann", "Bonjour !");
```

~10 lignes · 1 interface · 0 passe-partout

Les deux approches sont valides. La différence : avec LangChain4j-CDI, il n'y a pas de code de configuration à écrire. CDI découvre l'interface, crée le bean synthétique, résout les dépendances — tout en convention over configuration.

Double support d'extension CDI

LangChain4j-CDI fournit **les deux** types d'extensions, donc ça fonctionne sur **n'importe quel** serveur CDI :

TYPE D'EXTENSION	TRAITEMENT	SERVEURS
Portable Extension	Runtime	WildFly, Payara, GlassFish, Liberty
Build Compatible Extension	Compile-time	Quarkus, Helidon

Structure du projet

```
demo-project/
├─ pom.xml ← Parent POM (centralized versions)
├─ demo-1-ai-agent/ ← Injectable AI Agent
│   ├── base/ ← Skeleton with TODOs (your workspace)
│   └─ solution/ ← Complete reference
├─ demo-2-ft-telemetry/ ← Memory + RAG + Tools + FT + Telemetry
│   ├── base/
│   └─ solution/
├─ demo-3-mcp/ ← MCP Integration
│   ├── mcp-server/ ← Serveur MCP standalone (Helidon 4)
│   ├── base/
│   └─ solution/
└─ demo-4-guardrails/ ← Input & Output Guardrails
    ├── base/ ← Squelette avec TODOs (votre espace de travail)
    └─ solution/
```

Diapositives d'introduction

Présentation du contexte, des intervenants et du programme de l'atelier.

Dans la présentation : → slide suivant · 5 notes · 0 vue d'ensemble · Échap fermer le plein écran

Exercice 1 : Agent IA injectable

Message clé : « De 15 lignes de code passe-partout à 1 annotation + 1 propriété de configuration »

 OBJECTIF

Créez un chatbot IA simple en utilisant `@RegisterAIService`, injectez-le dans un point d'accès JAX-RS et testez-le. Ensuite, ajoutez des capacités de streaming et de vision multimodale.

```
cd demo-project/demo-1-ai-agent/base
# Ouvrez ce répertoire dans votre IDE
```

Fichiers clés

FICHIER	UTILITÉ
<code>ChatAssistant.java</code>	Interface du service IA (votre tâche principale)
<code>ChatAssistantStreaming.java</code>	Service IA en streaming
<code>ChatResource.java</code>	Point d'accès REST JAX-RS
<code>ImageAnalyzerServlet.java</code>	Servlet de vision multimodale
<code>microprofile-config.properties</code>	Configuration LLM

1 Créer l'interface du service IA

Ouvrez `ChatAssistant.java`. C'est une interface Java simple avec deux TODOs. Votre tâche : la transformer en agent IA injectable.

TODO 1 — Décommenter l'import

```
import dev.langchain4j.cdi.spi.RegisterAIService;
```

TODO 2 — Ajouter `@RegisterAIService` sur l'interface

```
@SuppressWarnings("CdiManagedBeanInconsistencyInspection")
@registerAIService(chatModelName = "my-model")
public interface ChatAssistant {

    @SystemMessage("""
        Tu es un skald viking qui raconte des blagues et des histoires drôles dans la grande salle.
        Tes blagues portent sur les guerriers maladroits, les raids qui tournent mal,
        les festins trop arrosés, les dieux nordiques et leurs facéties.
        Tes blagues sont courtes, percutantes et font rire tout le monde.
        Tu peux aussi raconter des anecdotes humoristiques sur la vie des vikings.
        """)
    String chat(@UserMessage String userMessage);
}
```

💡 CE QUI SE PASSE

- `@RegisterAIService` indique à l'extension CDI de créer un bean injectable pour cette interface
- `chatModelName = "my-model"` fait référence à la configuration du modèle dans `microprofile-config.properties`
- `@SystemMessage` définit la personnalité et le comportement de l'IA
- `@UserMessage` marque le paramètre comme entrée utilisateur

2 Câbler le point d'accès REST

Ouvrez `ChatResource.java`. Il contient une méthode `@PostConstruct init()` qui instancie manuellement les modèles et les services IA — c'est exactement ce que CDI remplace. Votre tâche : supprimer ce boilerplate et le remplacer par de l'injection.

TODO 1 — Supprimer les 4 imports inutiles en haut du fichier

```
// Supprimer ces 4 imports :
import dev.langchain4j.model.mistralai.MistralAiChatModel;
import dev.langchain4j.model.mistralai.MistralAiStreamingChatModel;
import dev.langchain4j.service.AiServices;
import jakarta.annotation.PostConstruct;
```

TODO 2 — Décommenter l'import CDI et remplacer les champs + `@PostConstruct`

```
import jakarta.inject.Inject;

@Path("/chat")
@ApplicationScoped
public class ChatResource {

    @Inject
    ChatAssistant assistant;

    @Inject
    ChatAssistantStreaming streamingAssistant;

    // Supprimer toute la méthode @PostConstruct init() ci-dessous

    @POST
    @Consumes(MediaType.TEXT_PLAIN)
    @Produces(MediaType.TEXT_PLAIN)
    public String chat(String message) {
        return assistant.chat(message);
    }

    // Le point d'accès /stream est déjà implémenté, rien à modifier ici.
}
```

✓ POINT CLÉ

Le code métier n'a **aucune dépendance sur LangChain4j**. Le seul import est votre propre interface. Changer de framework IA demain ? Changez l'implémentation, pas le code métier.

3 Configurer le modèle

Ouvrez `src/main/resources/META-INF/microprofile-config.properties`. La configuration est déjà prête — vérifiez simplement que le bon fournisseur est actif.

Par défaut, l'**Option A (Mistral AI)** est déjà décommentée :

```
# ---- Option A : Mistral AI (distant) ---- [ACTIVE PAR DÉFAUT]
dev.langchain4j.cdi.plugin.my-model.class=dev.langchain4j.model.mistralai.MistralAiChatModel
dev.langchain4j.cdi.plugin.my-model.config.api-key=${MISTRAL_API_KEY}
dev.langchain4j.cdi.plugin.my-model.config.model-name=mistral-small-latest

# ---- Option B : Ollama (local) ---- [commenté par défaut]
#dev.langchain4j.cdi.plugin.my-model.class=dev.langchain4j.model.ollama.OllamaChatModel
#dev.langchain4j.cdi.plugin.my-model.config.base-url=http://localhost:11434
#dev.langchain4j.cdi.plugin.my-model.config.model-name=ministral-3:3b

dev.langchain4j.cdi.plugin.my-model.config.temperature=0.9
dev.langchain4j.cdi.plugin.my-model.config.log-requests=true
```

OLLAMA LOCAL

Dans `microprofile-config.properties`, commentez les 3 lignes de l'**Option A** et décommentez les 3 lignes de l'**Option B**. Les propriétés communes (`temperature`, `log-requests` ...) restent inchangées.

CRITIQUE : LE PRÉFIXE .CONFIG.

Les propriétés du builder **doivent** utiliser le préfixe `.config.` : `dev.langchain4j.cdi.plugin.<name>.config.<prop>=value`. Sans `.config.`, la propriété est **ignorée silencieusement**.

Compiler et démarrer

```
mvn clean install
./target/server/bin/standalone.sh      # Linux / macOS
target\server\bin\standalone.bat       # Windows
```

```
# Dans un autre terminal :
curl -X POST -H "Content-Type: text/plain" \
  -d "Raconte-moi une blague viking !" \
  http://localhost:8080/demo-1/api/chat
```

4 Ajouter les réponses en streaming

Ouvrez `ChatAssistantStreaming.java`. Comme pour `ChatAssistant`, elle a deux TODOs :

TODO 1 — Décommenter l'import

```
import dev.langchain4j.cdi.spi.RegisterAIService;
```

TODO 2 — Ajouter `@RegisterAIService` sur l'interface

```
@SuppressWarnings("CdiManagedBeanInconsistencyInspection")
@RegisterAIService(streamingChatModelName = "my-streaming-model")
public interface ChatAssistantStreaming {

    @SystemMessage("""
        Tu es un skald viking, un conteur et poète de la grande salle.
        Tu chantes des récits épiques sur les batailles glorieuses, les raids audacieux,
        la bravoure des guerriers, les voyages en drakkar et la route vers le Valhalla.
        Tes chants sont rythmés, héroïques et pleins d'honneur.
        """)

    TokenStream chatStream(@UserMessage String userMessage);
}
```

💡 LE POINT D'ACCÈS SSE EST DÉJÀ LÀ

Dans `ChatResource.java`, le point d'accès `GET /stream` est **déjà implémenté**. Il sera fonctionnel dès que l'injection `streamingAssistant` sera en place (étape 2). Rien d'autre à modifier ici.

La configuration du modèle streaming est également déjà présente dans `microprofile-config.properties` — vérifiez que le bon fournisseur est actif (Option A ou B) :

```
# ---- Option A : Mistral AI (distant) ---- [ACTIVE PAR DÉFAUT]
dev.langchain4j.cdi.plugin.my-streaming-model.class=dev.langchain4j.model.mistralai.MistralAiStreamingChatModel
dev.langchain4j.cdi.plugin.my-streaming-model.config.api-key=${MISTRAL_API_KEY}
dev.langchain4j.cdi.plugin.my-streaming-model.config.model-name=mistral-small-latest

# ---- Option B : Ollama (local) ----
#dev.langchain4j.cdi.plugin.my-streaming-model.class=dev.langchain4j.model.ollama.OllamaStreamingChatModel
#dev.langchain4j.cdi.plugin.my-streaming-model.config.base-url=http://localhost:11434
#dev.langchain4j.cdi.plugin.my-streaming-model.config.model-name=minstral-3:3b
```

🏠 OLLAMA LOCAL

Commentez l'**Option A** du streaming model et décommentez l'**Option B** dans `microprofile-config.properties`.

5 Vision : analyse d'image

Ouvrez `ImageAnalyzerServlet.java` et injectez un modèle de vision :

```
@WebServlet("/uploadServlet")
@MultipartConfig
public class ImageAnalyzerServlet extends HttpServlet {

    @Inject
    @Named("vision-model")
    ChatModel visionModel;

    @Override
    protected void doPost(HttpServletRequest request,
        HttpServletResponse response) throws ... {
        Part file = request.getPart("file");

        UserMessage userMessage = UserMessage.from(
            TextContent.from("Décris cette image en détails."),
            ImageContent.from(encodeBase64(file.getInputStream()),
                file.getContentType())
        );

        ChatResponse answer = visionModel.chat(userMessage);
        response.getWriter().write(answer.aiMessage().text());
    }
}
```

Et la configuration du modèle de vision :

```
dev.langchain4j.cdi.plugin.vision-model.class=\
    dev.langchain4j.model.mistralai.MistralAiChatModel
dev.langchain4j.cdi.plugin.vision-model.config.api-key=${MISTRAL_API_KEY}
dev.langchain4j.cdi.plugin.vision-model.config.model-name=pixtral-large-latest
```

OLLAMA LOCAL

Même principe : commentez l'**Option A** du vision model et décommentez l'**Option B**. Le modèle Ollama déjà téléchargé (`ministral-3:3b`) sera utilisé.

**Tester et valider l'exercice 1**

```
# Chat standard
curl -X POST -H "Content-Type: text/plain" \
  -d "Raconte-moi une blague viking !" \
  http://localhost:8080/demo-1/api/chat

# Streaming (ouvrir dans le navigateur)
# http://localhost:8080/demo-1/stream.html

# Analyse d'image (ouvrir dans le navigateur)
# http://localhost:8080/demo-1/image.html

# Vérification de santé
curl http://localhost:8080/demo-1/api/chat/health
```

- ✓ Le chat standard renvoie une réponse du skald viking
- ✓ Le streaming affiche les tokens arrivant en temps réel
- ✓ Le téléchargement d'image renvoie une description générée par l'IA

BLOQUÉ ?

Passez à la solution : `cd ../solution && mvn clean install` puis `./target/server/bin/standalone.sh` (Linux/macOS) ou `target\server\bin\standalone.bat` (Windows)

Exercice 2 : Tolérance aux pannes et télémétrie

Message clé : « Les annotations MicroProfile fonctionnent sur les services IA parce qu'ils sont des beans CDI »

OBJECTIF

Commencez avec un guide d'expéditions viking fonctionnel qui possède déjà la mémoire, RAG et des outils. Votre tâche : ajouter des annotations MicroProfile Fault Tolerance pour le rendre prêt pour la production, puis l'observer avec OpenTelemetry.

Commencez par démarrer la stack Grafana LGTM (Loki + Grafana + Tempo + Mimir) dans un terminal dédié — elle sera prête quand vous en aurez besoin pour la télémétrie :

```
# Démarrer la stack LGTM complète (Loki + Grafana + Tempo + Mimir)
# avec le collecteur OpenTelemetry intégré — laisser tourner en arrière-plan
podman run -p 3000:3000 -p 4317:4317 -p 4318:4318 --rm -ti grafana/otel-lgtm

# Ou avec Docker
docker run -p 3000:3000 -p 4317:4317 -p 4318:4318 --rm -ti grafana/otel-lgtm
```

Puis lancez l'application dans un autre terminal :

```
cd demo-project/demo-2-ft-telemetry/base
mvn clean install
./target/server/bin/standalone.sh      # Linux / macOS
target\server\bin\standalone.bat      # Windows
```

Ouvrez votre navigateur vers <http://localhost:8080/demo-2> (<http://localhost:8080/demo-2>) pour tester l'application.

Ce qui fonctionne déjà

Le module base est pré-câblé avec ces fonctionnalités :



Mémoire de conversation

`@MemoryId` fournit une mémoire par session via `ChatMemoryProviderBean`. Chaque onglet du navigateur obtient sa propre conversation.



RAG

`ExpeditionDetailsProvider` ingeste les données d'expéditions viking dans un vector store en mémoire en utilisant les embeddings Mistral.



Outils

`VikingTools` est un bean CDI avec 5 méthodes `@Tool` : lister expéditions, enrôler guerrier, annuler, vérifier disponibilité, voir enrôlements.

Testez que tout fonctionne avant d'ajouter la tolérance aux pannes :

```
curl -X POST -H "Content-Type: text/plain" \
-H "X-Session-Id: test-123" \
-d "What expeditions are available?" \
http://localhost:8080/demo-2/api/chat
```

1 Ajouter @Retry

Ouvrez `ChatAssistant.java`. L'interface du service IA possède déjà `@RegisterAIService` avec la mémoire, le RAG et les outils. Ajoutez l'annotation `@Retry` :

```
import org.eclipse.microprofile.faulttolerance.Retry;

@registerAIService(chatModelName = "my-model",
    chatMemoryProviderName = "my-memory",
    contentRetrieverName = "my-rag",
    tools = VikingTools.class)
public interface ChatAssistant {

    @Retry(maxRetries = 3, delay = 1000)
    String chat(@MemoryId String sessionId,
        @UserMessage String message);
}
```

💡 POURQUOI ÇA FONCTIONNE

Comme `@RegisterAIService` crée un bean CDI, **tous les intercepteurs MicroProfile s'appliquent**. Le conteneur CDI enveloppe le proxy avec l'intercepteur Fault Tolerance — aucun code de liaison nécessaire.

2 Ajouter @Timeout et @Fallback

Empilez les annotations Fault Tolerance :

```
import org.eclipse.microprofile.faulttolerance.*;
import java.time.temporal.ChronoUnit;

@RegisterAIService(chatModelName = "my-model",
                  chatMemoryProviderName = "my-memory",
                  contentRetrieverName = "my-rag",
                  tools = VikingTools.class)
public interface ChatAssistant {

    @Retry(maxRetries = 3, delay = 1000)
    @Timeout(value = 30, unit = ChronoUnit.SECONDS)
    @Fallback(fallbackMethod = "chatFallback")
    String chat(@MemoryId String sessionId,
               @UserMessage String message);

    default String chatFallback(String sessionId, String message) {
        return "I'm sorry, our AI service is temporarily unavailable. "
            + "Please try again in a moment. "
            + "Our team has been notified.";
    }
}
```

3 Ajouter @CircuitBreaker

Ajoutez la dernière pièce — le circuit breaker empêche de surcharger un service défaillant :

```
@Retry(maxRetries = 3, delay = 1000)
@Timeout(value = 30, unit = ChronoUnit.SECONDS)
@Fallback(fallbackMethod = "chatFallback")
@CircuitBreaker(requestVolumeThreshold = 5, failureRatio = 0.5)
String chat(@MemoryId String sessionId,
           @UserMessage String message);
```

Rechargement à chaud : sauvegardez et exécutez `mvn package` (ou laissez votre IDE compiler automatiquement).

🔗 DÉCOMMENTER LA DÉPENDANCE FAULT TOLERANCE

Maintenant que les annotations sont en place, ouvrez `pom.xml` et décommentez la dépendance `langchain4j-cdi-fault-tolerance` (bloc marqué `TODO`). Sans elle, les annotations sont ignorées silencieusement.

```
<dependency>
  <groupId>dev.langchain4j.cdi.mp</groupId>
  <artifactId>langchain4j-cdi-fault-tolerance</artifactId>
</dependency>
```

✓ TESTER LA RÉSILIENCE

Simulez une panne : coupez le wifi pour simuler un souci réseau, puis envoyez des requêtes. Si vous utilisez Ollama, coupez simplement le serveur. Vous devriez voir les tentatives de retry, puis le message de fallback. Rétablissez la connexion (ou relancez Ollama) et le circuit breaker se rétablit automatiquement.

4 Ajouter l'observabilité OpenTelemetry

La stack Grafana LGTM est déjà lancée depuis le début de l'exercice. La configuration de télémétrie est déjà dans `microprofile-config.properties` :

```
# OpenTelemetry
otel.exporter.otlp.endpoint=http://localhost:4318
otel.service.name=demo-2-langchain4j-cdi
otel.traces.exporter=otlp
otel.metrics.exporter=otlp

# LangChain4j-CDI Telemetry Listeners
dev.langchain4j.cdi.plugin.my-model.config.listeners=\
    dev.langchain4j.cdi.telemetry.SpanChatModelListener,\
    dev.langchain4j.cdi.telemetry.MetricsChatModelListener
```

Envoyez quelques requêtes de chat, puis ouvrez <http://localhost:3000> (<http://localhost:3000>) (Grafana). Naviguez vers **Explore** → **Tempo** et recherchez le service `demo-2-langchain4j-cdi`.

Vous verrez des traces distribuées montrant :

- La latence des appels LLM
- Le nombre de tokens (entrée/sortie)
- Les appels d'outils et leur durée
- Les étapes de récupération RAG

✓ Tester et valider l'exercice 2

```
# Chat avec session
curl -X POST -H "Content-Type: text/plain" \
  -H "X-Session-Id: test-456" \
  -d "What expeditions are available?" \
  http://localhost:8080/demo-2/api/chat

# Enrôler un guerrier
curl -X POST -H "Content-Type: text/plain" \
  -H "X-Session-Id: test-456" \
  -d "Enroll Erik Bloodaxe for the first expedition" \
  http://localhost:8080/demo-2/api/chat

# Inspecter la mémoire
curl "http://localhost:8080/demo-2/api/chat/memory?sessionId=test-456"
```

- ✓ La mémoire conserve le contexte entre les messages d'une même session
- ✓ Le RAG récupère les informations pertinentes sur les expéditions
- ✓ Les outils exécutent les enrôlements, annulations et requêtes
- ✓ Le fallback répond gracieusement quand l'IA est indisponible
- ✓ Les traces apparaissent dans Grafana/Tempo

Exercice 3 : Intégration MCP

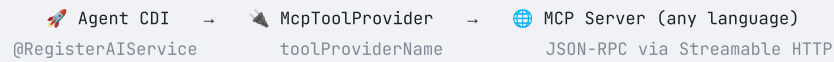
Message clé : « MCP est le JDBC de l'IA — vos agents communiquent avec n'importe quel serveur d'outils »

OBJECTIF

Connectez un agent IA à un serveur MCP (Model Context Protocol) externe. Le serveur MCP fournit des outils de lancer de runes, et votre agent IA anime un jeu de Hnefatafl au Grand Thing via le protocole.

Qu'est-ce que MCP ?

Le **Model Context Protocol** est un standard ouvert pour connecter les agents IA à des outils externes. Voyez-le comme un adaptateur universel : un seul protocole, n'importe quel serveur d'outils, n'importe quel langage.



0 Compiler et démarrer le serveur MCP

D'abord, compilez et démarrez le serveur MCP de dés externe :

```
# Compiler et démarrer le serveur MCP avec Helidon 4 (écoute sur le port 8090)
cd demo-project/demo-3-mcp/mcp-server
mvn clean package
java -jar target/casino-dice-roller.jar
```

Le serveur MCP est une application MicroProfile propulsée par **LangChain4j-CDI MCP Server** et **Helidon 4**. Il expose automatiquement un endpoint `/mcp` qui parle le protocole MCP (JSON-RPC 2.0 sur HTTP Streamable). L'outil `roll` génère des résultats de dés aléatoires pour le jeu viking.

💡 LAISSEZ-LE TOURNER

Laissez le serveur MCP tourner dans un terminal séparé. L'agent IA s'y connectera via l'endpoint MCP sur `localhost:8090/mcp`.

Tester avec MCP Inspector

Avant de connecter le serveur MCP à l'agent IA, vous pouvez vérifier qu'il fonctionne en utilisant le **MCP Inspector** — un outil de débogage visuel pour les serveurs MCP. Dans un nouveau terminal :

```
# Lancer le MCP Inspector
npx @modelcontextprotocol/inspector
```

Ceci ouvre une interface web (généralement sur `http://localhost:6274`). Configurez la connexion :

- **Transport Type** : `Streamable HTTP`
- **URL** : `http://localhost:8090/mcp`

Cliquez sur **Connect**, puis sur l'onglet **Tools** et sur l'outil `roll` pour l'inspecter et le tester interactivement :

MCP Inspector v0.21.1

Transport Type

Streamable HTTP

URL

http://localhost:8090/stream

Connection Type

Via Proxy

Server Entry

Servers File

Authentication

Configuration

Reconnect

Disconnect

Connected

default-server

Version: ROOT.war

Resources

Prompts

Tools

Tasks

Apps

Ping

Sampling

Elicitations

Roots

Auth

Metadata

Tools

List Tools

Clear

roll

Roll a number of dices and return the results

History

Clear

12. tools/call

roll

Roll a number of dices and return the results

Read-only

Destructive

Idempotent

Open-work

numberOfDice

3

Tool-specific Metadata:

No metadata pairs.

Run Tool

Copy Input

Tool Result: Success

[
 0: 2
 1: 4
 2: 5
]

Server Notifications

No notifications yet

 POINTS À VÉRIFIER

- Le statut de connexion affiche **Connected**
- L'outil `roll` apparaît dans la liste des Tools
- Mettez `numberOfDice` à `2`, cliquez sur **Run Tool**, et vérifiez que vous obtenez des résultats de dés

1 Créer le service IA du Jarl du Grand Thing

Ouvrez `HnefataflJarlAI.java` dans `demo-3-mcp/base/` :

```
@RegisterAIService(chatModelName = "mistral",
    toolProviderName = "mcp",
    chatMemoryProviderName = "my-memory")
public interface HnefataflJarlAI {

    @SystemMessage("""
        You are Ragnar the Skald, the Jarl hosting Hnefatafl
        at the Grand Thing of the Northern warriors.

        RULES: Cast 2 rune stones with roll(numberOfDice=2).
        OPENING CAST:
        - 7 or 11: Odin's Favour -- warrior WINS!
        - 2, 3 or 12: Curse of the Norns -- warrior LOSES!
        - Other: that number becomes THE MARKED RUNE.
        RUNE PHASE: hit the rune = WIN, roll 7 = Ragnarök = LOSE.

        REQUIRED FORMAT:
        RUNES: [X, Y]
        TOTAL: [sum]
        DESTIN: [what happened]

        Respond in French.
        """)
    String play(@UserMessage String playerAction);
}
```

 TOOLPROVIDERNAME

`toolProviderName = "mcp"` connecte cet agent au bean `McpToolProvider`, qui découvre automatiquement les outils de lancer de runes depuis le serveur MCP externe.

2 Configurer MCP via les propriétés

Ouvrez `microprofile-config.properties` et décommentez la configuration MCP :

```
# Chat Model
# ---- Option A : Mistral AI (distant) ----
dev.langchain4j.cdi.plugin.mistral.class=dev.langchain4j.model.mistralai.MistralAiChatModel
dev.langchain4j.cdi.plugin.mistral.config.api-key=${MISTRAL_API_KEY}
dev.langchain4j.cdi.plugin.mistral.config.model-name=mistral-small-latest

# MCP Transport (Streamable HTTP vers le serveur MCP de dés)
dev.langchain4j.cdi.plugin.ssetransport.class=\
    dev.langchain4j.mcp.client.transport.http.StreamableHttpMcpTransport
dev.langchain4j.cdi.plugin.ssetransport.config.url=http://localhost:8090/mcp
dev.langchain4j.cdi.plugin.ssetransport.config.logRequests=true
dev.langchain4j.cdi.plugin.ssetransport.config.logResponses=true

# MCP Client
dev.langchain4j.cdi.plugin.mcpclient.class=dev.langchain4j.mcp.client.DefaultMcpClient
dev.langchain4j.cdi.plugin.mcpclient.config.transport=lookup:@ssetransport

# MCP Tool Provider (nommé "mcp" pour @RegisterAIService)
dev.langchain4j.cdi.plugin.mcp.class=dev.langchain4j.mcp.McpToolProvider
dev.langchain4j.cdi.plugin.mcp.config.mcpClients=lookup:@mcpclient
```

OLLAMA LOCAL

Même principe : commentez l'**Option A** du Chat Model et décommentez l'**Option B**. La config MCP (transport, client, tool provider) reste identique.

LE PATTERN LOOKUP:

`Lookup:@ssetransport` référence un autre bean CDI par son nom de configuration. C'est ainsi que vous câblez les beans entre eux uniquement via la configuration — aucun code Java nécessaire pour la plomberie.

3 Câbler le point d'accès REST

Ouvrez `GameResource.java` :

```
@Path("/game")
@ApplicationScoped
public class GameResource {

    @Inject
    HnefataflJarlAI gameMaster;

    @POST
    @Path("/play")
    @Consumes(MediaType.TEXT_PLAIN)
    @Produces(MediaType.TEXT_PLAIN)
    public String play(String playerAction) {
        return gameMaster.play(playerAction);
    }

    @GET
    @Path("/start")
    @Produces(MediaType.TEXT_PLAIN)
    public String start() {
        return gameMaster.play(
            "Salut ! Je suis prêt pour une partie de Hnefatafl !");
    }
}
```

✓ Tester et valider l'exercice 3

```
# Assurez-vous que le serveur MCP tourne sur le port 8090 !

# Compiler et démarrer
cd demo-project/demo-3-mcp/base
mvn clean install
./target/server/bin/standalone.sh      # Linux / macOS
target\server\bin\standalone.bat      # Windows

# Démarrer une partie
curl http://localhost:8080/demo-3/api/game/start

# Lancer les dés
curl -X POST -H "Content-Type: text/plain" \
  -d "Lance les runes !" \
  http://localhost:8080/demo-3/api/game/play

# Ouvrir l'interface web
# http://localhost:8080/demo-3/
```

- ✓ L'agent IA appelle l'outil `roll` du serveur MCP via Streamable HTTP
- ✓ Les résultats des runes sont retournés au format RUNES: [X, Y]
- ✓ Ragnar le Skald annonce les victoires, malédictions et runes marquées dans le style viking
- ✓ L'interface web affiche des dés visuels avec des points animés

🎉 FÉLICITATIONS !

Vous avez construit trois applications propulsées par l'IA avec LangChain4j-CDI : un skald viking, un guide d'expéditions résilient, et un jeu de Hnefatafl intégré via MCP. Le tout avec des patterns Jakarta EE standards.

Exercice 4 : Guardrails — Garde-fous pour votre agent

Message clé : « Les guardrails sont des beans CDI — ils bénéficient de l'injection de dépendances comme tous vos composants »

🚩 OBJECTIF

Ajoutez quatre guardrails autour de votre skald viking : deux sur l'entrée (valider la requête avant d'appeler le LLM) et deux sur la sortie (valider la réponse avant de la renvoyer). L'agent n'accepte que le français et refuse toute mention de races fantastiques.

```
cd demo-project/demo-4-guardrails/base
# Ouvrir ce répertoire dans votre IDE
```

Concept : InputGuardrail vs OutputGuardrail

InputGuardrail

S'exécute **avant** d'appeler le LLM. Si le guardrail échoue, le LLM n'est jamais appelé — économie de temps et de tokens.

```
InputGuardrailResult validate(
    UserMessage userMessage)
```

OutputGuardrail

S'exécute **après** la réponse du LLM. Si le guardrail échoue, une exception est levée — le code appelant doit la gérer.

```
OutputGuardrailResult validate(
    AiMessage responseFromLLM)
```


Fichiers clés

FICHIER	RÔLE	TODO
<code>SkaIdAssistant.java</code>	Interface du service IA	Ajouter <code>@RegisterAIService</code> avec les noms de guardrails
<code>FrenchInputGuardrail.java</code>	Vérifie que la requête est en français	Nommer, initialiser, implémenter
<code>FrenchOutputGuardrail.java</code>	Vérifie que le chant est en français	Nommer, initialiser, implémenter
<code>NoFantasyRacesInputGuardrail.java</code>	Rejette les mentions de nains/elfes en entrée	Nommer, implémenter
<code>NoFantasyRacesOutputGuardrail.java</code>	Rejette les mentions de nains/elfes en sortie	Nommer, implémenter
<code>ChatResource.java</code>	Point d'accès REST (déjà câblé)	Rien — gère déjà les exceptions guardrail

⚠ PRÉREQUIS

Cet exercice utilise **Mistral AI** (comme l'exercice 1). Assurez-vous que `MISTRAL_API_KEY` est exporté dans votre terminal.

1 Nommer les guardrails avec `@Named`

Les guardrails sont des beans CDI référencés par leur nom dans `@RegisterAIService`. Ajoutez `@Named` sur chaque classe de guardrail.

FrenchInputGuardrail.java

```
import jakarta.inject.Named;

@ApplicationScoped
@Named("french-input") // - Ajouter cette annotation
public class FrenchInputGuardrail implements InputGuardrail {
    // ...
}
```

FrenchOutputGuardrail.java

```
@ApplicationScoped
@Named("french-output")
public class FrenchOutputGuardrail implements OutputGuardrail {
    // ...
}
```

NoFantasyRacesInputGuardrail.java

```
@ApplicationScoped
@Named("fantasy-input")
public class NoFantasyRacesInputGuardrail implements InputGuardrail {
    // ...
}
```

NoFantasyRacesOutputGuardrail.java

```
@ApplicationScoped
@Named("fantasy-output")
public class NoFantasyRacesOutputGuardrail implements OutputGuardrail {
    // ...
}
```

💡 POURQUOI @NAMED ?

LangChain4j-CDI résout les guardrails par leur nom CDI au moment du démarrage. Le nom dans `@Named("...")` doit correspondre exactement aux chaînes dans `inputGuardrailNames` et `outputGuardrailNames`.

2 Initialiser le détecteur de langue (Apache Tika)

Les guardrails de langue (`FrenchInputGuardrail` et `FrenchOutputGuardrail`) utilisent Apache Tika pour détecter automatiquement la langue d'un texte. Initialisez le détecteur dans `@PostConstruct` :

```
import org.apache.tika.langdetect.optimaize.OptimaizeLangDetector;
import org.apache.tika.language.detect.LanguageDetector;

@ApplicationScoped
@Named("french-input")
public class FrenchInputGuardrail implements InputGuardrail {

    private LanguageDetector detector;

    @PostConstruct
    void init() {
        detector = new OptimaizeLangDetector().loadModels();
        //      ↑ Charge les modèles de détection de langue depuis le classpath
    }

    // ...
}
```

Faites de même dans `FrenchOutputGuardrail` .

💡 THREAD-SAFETY

Le bean est `@ApplicationScoped` : une seule instance partagée entre toutes les requêtes. Le `LanguageDetector` d'Apache Tika n'est **pas thread-safe** — vous devrez utiliser `synchronized (detector) { ... }` lors de la détection.

3 Implémenter la méthode `validate()`

FrenchInputGuardrail — accepter uniquement le français

```
@Override
public InputGuardrailResult validate(UserMessage userMessage) {
    String text = userMessage.singleText();
    LanguageResult result;
    synchronized (detector) {
        result = detector.detect(text);
    }
    // Benefit of the doubt : si Tika n'est pas sûr (texte trop court), on laisse passer
    if (result.isReasonablyCertain() && !"fr".equals(result.getLanguage())) {
        return failure("Seul le français est accepté ! Parlez français pour invoquer le skald.");
    }
    return success();
}
```

FrenchOutputGuardrail — vérifier que le chant est en français

```
@Override
public OutputGuardrailResult validate(AiMessage responseFromLLM) {
    String text = responseFromLLM.text();
    LanguageResult result;
    synchronized (detector) {
        result = detector.detect(text);
    }
    // Pour la sortie, on est strict : si pas certain ou pas français - échec
    if (!result.isReasonablyCertain() || !"fr".equals(result.getLanguage())) {
        return failure("Le chant doit être en français ! Le skald doit chanter dans la langue de Molière.");
    }
    return success();
}
```

💡 DIFFÉRENCE ENTRÉE / SORTIE

Pour l'**entrée** : on applique le *benefit of the doubt* — un seul mot comme « Odin » n'est pas détectable avec certitude, donc on accepte.
 Pour la **sortie** : le chant est long, Tika est fiable — on rejette si non certain ou si ce n'est pas du français.

NoFantasyRacesInputGuardrail — rejeter les mots interdits en entrée

```
private static final Set<String> FORBIDDEN_WORDS =
    Set.of("nain", "nains", "elfe", "elf", "elfes");

@Override
public InputGuardrailResult validate(UserMessage userMessage) {
    String text = userMessage.singleText().toLowerCase();
    for (String word : FORBIDDEN_WORDS) {
        if (text.contains(word)) {
            return failure("Les Vikings ne connaissent ni les nains ni les elfes ! "
                + "Gardez votre requête purement nordique.");
        }
    }
    return success();
}
```

NoFantasyRacesOutputGuardrail — rejeter les mots interdits en sortie

```
@Override
public OutputGuardrailResult validate(AiMessage responseFromLLM) {
    String text = responseFromLLM.text().toLowerCase();
    for (String word : FORBIDDEN_WORDS) {
        if (text.contains(word)) {
            return failure("Le chant du skald ne doit pas évoquer les nains ni les elfes ! "
                + "C'est une saga viking pure.");
        }
    }
    return success();
}
```

4 Câbler les guardrails dans @RegisterAIService

Ouvrez `SkaldAssistant.java` et décommentez / remplacez le TODO par l'annotation complète :

```
import dev.langchain4j.cdi.spi.RegisterAIService;
import dev.langchain4j.service.SystemMessage;
import dev.langchain4j.service.UserMessage;

@RegisterAIService(
    chatModelName = "my-model",
    inputGuardrailNames = {"french-input", "fantasy-input"},
    outputGuardrailNames = {"french-output", "fantasy-output"})
public interface SkaldAssistant {

    @SystemMessage("""
        Tu es un skald viking qui compose des chants épiques en FRANÇAIS.
        Le chant doit célébrer les thèmes nordiques héroïques : batailles, raids,
        Valhalla, Thor, Odin, drakkars.
        Format : 3-4 couplets avec un refrain puissant.
        Ton : épique, héroïque, guerrier.
        """)
    String composeSong(@UserMessage String request);
}
```

💡 ORDRE D'EXÉCUTION

Les guardrails sont exécutés dans l'ordre des tableaux. Ici, `french-input` est évalué avant `fantasy-input`. Si le premier échoue, le second n'est pas évalué. Le LLM n'est appelé que si **tous** les guardrails d'entrée passent.

Compiler et démarrer

```
mvn clean install
./target/server/bin/standalone.sh      # Linux / macOS
target\server\bin\standalone.bat       # Windows
```

⚠ CONFIGURATION DÉJÀ FOURNIE

Le fichier `microprofile-config.properties` est déjà configuré avec le modèle Mistral AI. Assurez-vous que `MISTRAL_API_KEY` est défini dans votre environnement.

✓ Tester et valider l'exercice 4

Via curl

```
# ✓ Requête valide en français
curl -X POST -H "Content-Type: text/plain" \
  -d "Compose un chant sur la gloire d'Odin et les batailles vikings" \
  http://localhost:8080/demo-4/api/song

# ✗ InputGuardrail langue : requête en anglais rejetée
curl -X POST -H "Content-Type: text/plain" \
  -d "Sing me a heroic Viking song about Odin and his ravens" \
  http://localhost:8080/demo-4/api/song

# ✗ InputGuardrail races : mention de nains rejetée
curl -X POST -H "Content-Type: text/plain" \
  -d "Compose un chant sur les nains qui forgeaient des armes vikings" \
  http://localhost:8080/demo-4/api/song

# Vérification santé
curl http://localhost:8080/demo-4/api/song/health
```

Via l'interface web

Ouvrez <http://localhost:8080/demo-4/> (<http://localhost:8080/demo-4/>) pour accéder à l'interface interactive qui affiche les guardrails actifs et leurs messages d'erreur.

- ✓ Une requête en français sur un thème nordique produit un chant épique
- ✓ Une requête en anglais retourne « *Seul le français est accepté !* »
- ✓ Une requête mentionnant des nains ou des elfes est rejetée
- ✓ Un seul mot ambigu comme « Odin » passe (benefit of the doubt)
- ✓ Le code HTTP 400 est retourné avec le message du guardrail qui a échoué

💡 BLOQUÉ ?

Passez à la solution : `cd ../solution && mvn clean install` puis `./target/server/bin/standalone.sh` (Linux/macOS) ou `target\server\bin\standalone.bat` (Windows)

🎉 FÉLICITATIONS !

Vous avez construit quatre applications alimentées par l'IA avec LangChain4j-CDI. Passez à l'exercice 5 pour découvrir l'orchestration multi-agents avec le protocole A2A !

Exercice 5 : A2A — Orchestration multi-agents

Message clé : « MCP connecte les agents aux outils. A2A connecte les agents *entre eux* — les microservices version IA »

📌 OBJECTIF

Construisez trois services WildFly indépendants communiquant via le protocole A2A (Agent-to-Agent) : un **Creative Writer** qui génère des histoires, un **Style Scorer** qui évalue le style, et un **Orchestrateur** qui coordonne le tout avec une boucle de révision automatique.

```
cd demo-project/demo-5-a2a/base
# Ouvrir ce répertoire dans votre IDE
```

Concept : MCP vs A2A

MCP (Model Context Protocol)

Un agent appelle des **outils** externes (bases de données, APIs, fichiers). Communication agent → outil.

Agent → MCP Server → Tool

A2A (Agent-to-Agent)

Un agent **délègue** à d'autres agents spécialisés, chacun sur son propre serveur. Communication agent ↔ agent.

Orchestrateur ↔ Agent A2A ↔ Agent A2A

Architecture de Story Forge

L'application **Story Forge** orchestre trois services :

- 1. **Creative Writer** (port 8080) — Forge une saga nordique courte sur un sujet donné
- 2. **Style Scorer** (port 8081) — Évalue l'adéquation de la saga avec un style (score 0.0–1.0)
- 3. **Orchestrateur** (port 8082) — Coordonne le pipeline : écriture → boucle (scorer + éditeur local) jusqu'à score ≥ 0.8

Fichiers clés

a2a-creative-writer

FICHIER	RÔLE	TODO
CreativeWriter.java	Service IA LangChain4j	Ajouter @RegisterAIService et @UserMessage
CreativeWriterAgentCardProducer.java	Décrit l'agent au réseau A2A	Déjà implémenté — observer
CreativeWriterExecutorProducer.java	Reçoit et traite les requêtes A2A	Déjà implémenté — observer

a2a-style-scorer

FICHIER	RÔLE	TODO
StyleScorer.java	Service IA LangChain4j	Ajouter @RegisterAIService et @UserMessage
StyleScorerAgentCardProducer.java	Décrit l'agent au réseau A2A	Déjà implémenté — observer
StyleScorerExecutorProducer.java	Reçoit et traite les requêtes A2A	Déjà implémenté — observer

a2a-orchestrator

FICHIER	RÔLE	TODO
A2ACreativeWriter.java	Proxy CDI vers le Creative Writer distant	Ajouter @RegisterA2AAgent(a2aServerUrl=...)
A2AStyleScorer.java	Proxy CDI vers le Style Scorer distant	Ajouter @RegisterA2AAgent(a2aServerUrl=...)
StyleEditor.java	Agent local de réécriture (Ollama)	Ajouter @RegisterSimpleAgent(chatModelName="ollama")
StyleReviewLoop.java	Boucle de révision déclarative	Ajouter @RegisterLoopAgent(exitConditionName=...)
ScoreExitCondition.java	Condition de sortie CDI (Predicate<AgenticScope>)	Implémenter le test score ≥ 0.8
StyLEDWriter.java	Pipeline SEQUENCE complet	Ajouter @RegisterSequenceAgent(subAgentNames=...)
OrchestratorService.java	Injecte et appelle le pipeline	Injecter StyLEDWriter et appeler writeStoryWithStyle()
StyLEDWriterEndpoint.java	Point d'accès REST	Extraire le score depuis le AgenticScope

⚠ PRÉREQUIS

Cet exercice utilise **Ollama** avec le modèle `qwen2.5:7b`. Assurez-vous qu'Ollama tourne (`ollama serve`) et que le modèle est téléchargé (`ollama pull qwen2.5:7b`).

1 Implémenter le service IA `CreativeWriter`

Ouvrez `a2a-creative-writer/src/main/java/com/example/demo5/writer/CreativeWriter.java` et complétez l'interface :

```
package com.example.demo5.writer;

import dev.langchain4j.cdi.spi.RegisterAIService;
import dev.langchain4j.service.UserMessage;
import dev.langchain4j.service.V;
import jakarta.enterprise.context.ApplicationScoped;

@RegisterAIService(chatModelName = "ollama", scope = ApplicationScoped.class)
public interface CreativeWriter {

    @UserMessage("""
        You are a Norse skald, a Viking bard who crafts mighty sagas of glory, battle, and adventure.
        Forge a short saga no more than 3 sentences around the given topic, in the spirit of the Viking age.
        Return only the saga in English and nothing else.
        The topic is {{topic}}.
        """)
    String generateStory(@V("topic") String topic);
}
```

💡 @REGISTERAISERVICE

Comme dans les exercices précédents, `@RegisterAIService` transforme l'interface en bean CDI injectable. Le `chatModelName = "ollama"` référence la configuration dans `microprofile-config.properties`.

2 Implémenter le service IA `StyleScorer`

Ouvrez `a2a-style-scorer/src/main/java/com/example/demo5/scorer/StyleScorer.java` et complétez l'interface :

```
package com.example.demo5.scorer;

import dev.langchain4j.cdi.spi.RegisterAIService;
import dev.langchain4j.service.UserMessage;
import dev.langchain4j.service.V;
import jakarta.enterprise.context.ApplicationScoped;

@RegisterAIService(chatModelName = "ollama", scope = ApplicationScoped.class)
public interface StyleScorer {

    @UserMessage("""
        You are a seasoned Norse skald elder who judges sagas by the fire of a longhouse.
        Give a score between 0.0 and 1.0 for the following saga
        based on how well it captures the '{{style}}' style.
        Return only the score and nothing else.

        The saga is: "{{story}}"
        """)
    double scoreStyle(@V("story") String story, @V("style") String style);
}
```

💡 TYPE DE RETOUR `DOUBLE`

LangChain4j parse automatiquement la réponse du LLM en `double`. Le prompt demande explicitement un score numérique pour faciliter le parsing.

3 Observer l'AgentCard et l'AgentExecutor (déjà implémentés)

Chaque agent A2A a besoin de deux composants CDI : une **AgentCard** (qui décrit l'agent au réseau) et un **AgentExecutor** (qui traite les requêtes). Ces fichiers sont **déjà implémentés** dans le module `base/` — prenez un moment pour comprendre leur structure.

AgentCard — Creative Writer

Ouvrez `CreativeWriterAgentCardProducer.java` et observez la méthode `agentCard()` :

```
@Produces
@PublicAgentCard
public AgentCard agentCard() {
    return AgentCard.builder()
        .name("Creative Writer")
        .description("Generate a Norse saga based on the given topic")
        .url(serverUrl)
        .version("1.0.0")
        .capabilities(AgentCapabilities.builder()
            .streaming(true)
            .pushNotifications(false)
            .build())
        .defaultInputModes(Collections.singletonList("text"))
        .defaultOutputModes(Collections.singletonList("text"))
        .skills(Collections.singletonList(AgentSkill.builder()
            .id("creative_writer")
            .name("Creative Writer")
            .description("Generate a Norse saga based on the given topic")
            .tags(Collections.singletonList("writing"))
            .build()))
        .preferredTransport(TransportProtocol.JSONRPC.asString())
        .supportedInterfaces(List.of(
            new AgentInterface(TransportProtocol.JSONRPC.asString(), serverUrl)))
        .build();
}
```

AgentExecutor — Creative Writer

Ouvrez `CreativeWriterExecutorProducer.java` et observez la méthode `execute()` :

```
@Override
public void execute(RequestContext context, AgentEmitter emitter) throws A2AError {

    // ÉTAPE 1 : Démarrer le traitement
    emitter.startWork();

    // ÉTAPE 2 : Extraire le texte du message A2A
    String userMessage = extractTextFromMessage(context.getMessage());

    // ÉTAPE 3 : Appeler le service IA
    String response = creativeWriter.generateStory(userMessage);

    // ÉTAPE 4 : Répondre et compléter
    TextPart responsePart = new TextPart(response);
    emitter.addArtifact(List.of(responsePart), null, null, null);
    emitter.complete();
}
```

Style Scorer — même pattern

Observez également `StyleScorerAgentCardProducer.java` (skill id : `"style_scorer"`) et `StyleScorerExecutorProducer.java` (extrait deux arguments : `story` et `style`) — déjà implémentés.

💡 PATTERN SERVEUR A2A

Un serveur A2A Jakarta EE se résume à trois composants CDI : le `@RegisterAIService` (logique IA), un `@Produces @PublicAgentCard` (description), et un `@Produces AgentExecutor` (pont entre A2A et le service IA).

4 Câbler l'orchestrateur agentic

Étape 4a — Annoter les proxies A2A (`A2ACreativeWriter.java` , `A2AStyleScorer.java`)

Chaque interface devient un bean CDI grâce à `@RegisterA2AAgent`. L'URL de l'agent est lue depuis la configuration MicroProfile :

```
import dev.langchain4j.cdi.spi.RegisterA2AAgent;
import dev.langchain4j.service.V;

@RegisterA2AAgent(
    name = "creative-writer",
    a2aServerUrl = "${a2a.creative-writer.url}", // lu dans microprofile-config.properties
    outputKey = "story",
    description = "Generate a Norse saga based on the given topic")
public interface A2ACreativeWriter {
    String generateStory(@V("topic") String topic);
}

@RegisterA2AAgent(
    name = "style-scorer",
    a2aServerUrl = "${a2a.style-scorer.url}",
    outputKey = "score",
    description = "Score a saga based on how well it captures a given style")
public interface A2AStyleScorer {
    double scoreStyle(@V("story") String story, @V("style") String style);
}
```

Étape 4b — Annoter l'éditeur local (`StyleEditor.java`)

```
import dev.langchain4j.cdi.spi.RegisterSimpleAgent;
import dev.langchain4j.service.UserMessage;
import dev.langchain4j.service.V;

@RegisterSimpleAgent(
    name = "style-editor",
    description = "Reforge a saga to better capture a given style",
    outputKey = "story",
    chatModelName = "ollama")
public interface StyleEditor {
    @UserMessage("""
        You are a master Norse skald who shapes and tempers sagas like a smith forges steel.
        Rewrite the following saga to better honor and embody the {{style}} style.
        Return only the saga and nothing else.
        The saga is "{{story}}".
        """)
    String editStory(@V("story") String story, @V("style") String style);
}
```

Étape 4c — Déclarer la boucle (`StyleReviewLoop.java`) et la condition de sortie (`ScoreExitCondition.java`)

La boucle est entièrement déclarative grâce à `@RegisterLoopAgent`. La condition de sortie est un bean CDI `@Named` qui implémente `Predicate<AgenticScope>` :

```
import dev.langchain4j.cdi.spi.RegisterLoopAgent;
import dev.langchain4j.service.V;

@RegisterLoopAgent(
    name = "style-review-loop",
    subAgentNames = {"style-scorer", "style-editor"},
    maxIterations = 5,
    exitConditionName = "scoreExitCondition",
    exitConditionDescription = "Exit when the story score reaches 0.8 or higher")
public interface StyleReviewLoop {
    String refineStory(@V("story") String story, @V("style") String style);
}
```

```
import dev.langchain4j.agentic.scope.AgentScope;
import jakarta.enterprise.context.ApplicationScoped;
import jakarta.inject.Named;
import java.util.function.Predicate;

@ApplicationScoped
@Named("scoreExitCondition")
public class ScoreExitCondition implements Predicate<AgentScope> {

    @Override
    public boolean test(AgentScope scope) {
        Object raw = scope.readState("score");
        if (raw == null) return false;
        if (raw instanceof Number n) return n.doubleValue() ≥ 0.8;
        try { return Double.parseDouble(raw.toString()) ≥ 0.8; }
        catch (NumberFormatException e) { return false; }
    }
}
```

Étape 4d — Annoter le pipeline (`StyledWriter.java`), simplifier `OrchestratorService.java` et mettre à jour `StyledWriterEndpoint.java`

```
import dev.langchain4j.agentic.scope.AgentScopeAccess;
import dev.langchain4j.agentic.scope.ResultWithAgentScope;
import dev.langchain4j.cdi.spi.RegisterSequenceAgent;
import dev.langchain4j.service.V;

@RegisterSequenceAgent(
    subAgentNames = {"creative-writer", "style-review-loop"},
    outputKey = "story")
public interface StyledWriter extends AgentScopeAccess {
    ResultWithAgentScope<String> writeStoryWithStyle(
        @V("topic") String topic, @V("style") String style);
}
```

```
import dev.langchain4j.agentic.scope.ResultWithAgentScope;
import jakarta.enterprise.context.ApplicationScoped;
import jakarta.inject.Inject;

@ApplicationScoped
public class OrchestratorService {

    @Inject
    StyledWriter styledWriter;

    public ResultWithAgentScope<String> writeStyledStory(String topic, String style) {
        return styledWriter.writeStoryWithStyle(topic, style);
    }
}
```

Enfin, mettez à jour `StyledWriterEndpoint.java` pour extraire le score depuis le `AgentScope` :

```
import dev.langchain4j.agentic.scope.AgentScope;
import dev.langchain4j.agentic.scope.ResultWithAgentScope;

// Dans la méthode writeStyledStory(), remplacez :
// String story = orchestratorService.writeStyledStory(topic, style);
// par :
ResultWithAgentScope<String> result = orchestratorService.writeStyledStory(topic, style);
String story = result.result();
AgentScope scope = result.agentScope();
double score = scope.readState("score", 0.0);

// Ajoutez "score" dans le JSON :
String json = ""
    {
        "story": %s,
        "score": %s,
        "topic": %s,
        "style": %s
    }
    """.formatted(jsonString(story), score, jsonString(topic), jsonString(style));
```

💡 ANNOTATIONS DÉDIÉES VS BUILDERS MANUELS

Avec les annotations `@RegisterA2AAgent`, `@RegisterSequenceAgent`, `@RegisterLoopAgent` et `@RegisterSimpleAgent`, LangChain4j-CDI enregistre automatiquement chaque agent comme bean CDI. L'URL `${a2a.creative-writer.url}` est résolue via `MicroProfile Config` au démarrage. La condition de sortie de la boucle est un bean CDI `@Named` référencé par `exitConditionName` — plus besoin de builders programmatiques.

Démarrer les trois services

Chaque service tourne sur un port différent. Ouvrez **trois terminaux** :

```
# Terminal 1 : Creative Writer (port 8080)
cd demo-project/demo-5-a2a/base/a2a-creative-writer
mvn clean install
./target/server/bin/standalone.sh           # Linux / macOS
target\server\bin\standalone.bat           # Windows

# Terminal 2 : Style Scorer (port 8081)
cd demo-project/demo-5-a2a/base/a2a-style-scorer
mvn clean install
./target/server/bin/standalone.sh -Djboss.socket.binding.port-offset=1 # Linux / macOS
target\server\bin\standalone.bat -Djboss.socket.binding.port-offset=1 # Windows

# Terminal 3 : Orchestrateur (port 8082)
cd demo-project/demo-5-a2a/base/a2a-orchestrator
mvn clean install
./target/server/bin/standalone.sh -Djboss.socket.binding.port-offset=2 # Linux / macOS
target\server\bin\standalone.bat -Djboss.socket.binding.port-offset=2 # Windows
```

⚠️ ORDRE DE DÉMARRAGE

Démarrez le Creative Writer et le Style Scorer **avant** l'Orchestrateur. L'orchestrateur se connecte aux agents A2A au démarrage.

✓ Tester et valider l'exercice 5

Via curl

```
# Forger une saga avec un style
curl "http://localhost:8082/api/styled-story?topic=Erik+le+Rouge+traverse+les+mers&style=epique"

# Autre exemple
curl "http://localhost:8082/api/styled-story?topic=la+bataille+de+Ragnarok&style=tragique"
```

Via l'interface web

Ouvrez <http://localhost:8082> (<http://localhost:8082>) pour accéder à l'interface Story Forge. Saisissez un sujet et un style, puis observez la génération.

- ✓ Les trois services WildFly démarrent sans erreur (ports 8080, 8081, 8082)
- ✓ L'interface web Story Forge s'affiche sur le port 8082
- ✓ Une requête produit une saga avec un score de style
- ✓ Les logs montrent la boucle de révision (scorer → editor → scorer...)
- ✓ Le score final est ≥ 0.8 (ou max 5 itérations atteintes)

💡 BLOQUÉ ?

Passez à la solution : utilisez les modules sous `demo-5-a2a/solution/` au lieu de `base/`.

🎉 FÉLICITATIONS !

Vous avez construit cinq applications alimentées par l'IA avec LangChain4j-CDI : un skald viking, un guide d'expéditions résilient, un jeu de dés avec MCP, un skald avec guardrails, et maintenant un pipeline multi-agents via A2A. Passez à l'exercice 6 pour remplacer la boucle explicite par un superviseur LLM avec `@RegisterSupervisorAgent` !

Exercice 6 : Supervisor Agent — orchestration pilotée par le LLM

Message clé : « Avec `@RegisterLoopAgent`, vous écrivez la condition de sortie. Avec `@RegisterSupervisorAgent`, le *modèle* décide quand la qualité est atteinte. »

📌 OBJECTIF

Reprenez l'architecture Story Forge de l'exercice 5 et remplacez uniquement l'orchestrateur. La boucle `@RegisterLoopAgent` + `ScoreExitCondition` disparaissent au profit d'un unique `@RegisterSupervisorAgent` : le LLM lit la story, appelle le scorer et l'éditeur, et décide lui-même quand le score est suffisant.

```
cd demo-project/demo-6-supervisor/base
# Ouvrir ce répertoire dans votre IDE
```

Différence clé : Loop vs Supervisor

Demo 5 — @RegisterLoopAgent

Condition de sortie codée en dur dans un bean CDI. Le code décide quand s'arrêter.

```
@RegisterLoopAgent(exitConditionName = "scoreExitCondition")
// + ScoreExitCondition implements Predicate<AgenticScope>
```

Demo 6 — @RegisterSupervisorAgent

Le LLM lit le `supervisorContext` et décide lui-même quand s'arrêter. Plus de Predicate.

```
@RegisterSupervisorAgent(supervisorContext = "...")
// Pas de ScoreExitCondition !
```

AgenticScope — évolution au fil du pipeline

Tous les agents partagent un unique `AgenticScope` — un espace clé/valeur mutable qui relie chaque agent du pipeline. Trois mécanismes régissent les échanges :

① Écriture par @V

Quand vous appelez une méthode d'agent avec des paramètres `@V("key")`, le framework écrit chaque valeur dans le scope *avant* d'invoquer l'agent. Les sous-agents du superviseur lisent ensuite leurs entrées depuis ce même scope.

```
// supervisor.refineStory(story, style)
// → scope["story"] = story
// → scope["style"] = style
// Les sous-agents liront ces clés
// automatiquement via le scope.
```

② Écriture par outputKey

Quand un agent déclare un `outputKey`, son résultat est automatiquement persisté dans le scope après exécution. Les agents suivants peuvent ainsi réutiliser ce résultat sans paramètre explicite.

```
// style-scorer (outputKey="score")
// → scope["score"] = 0.35

// style-editor (outputKey="story")
// → scope["story"] = "saga v2..."

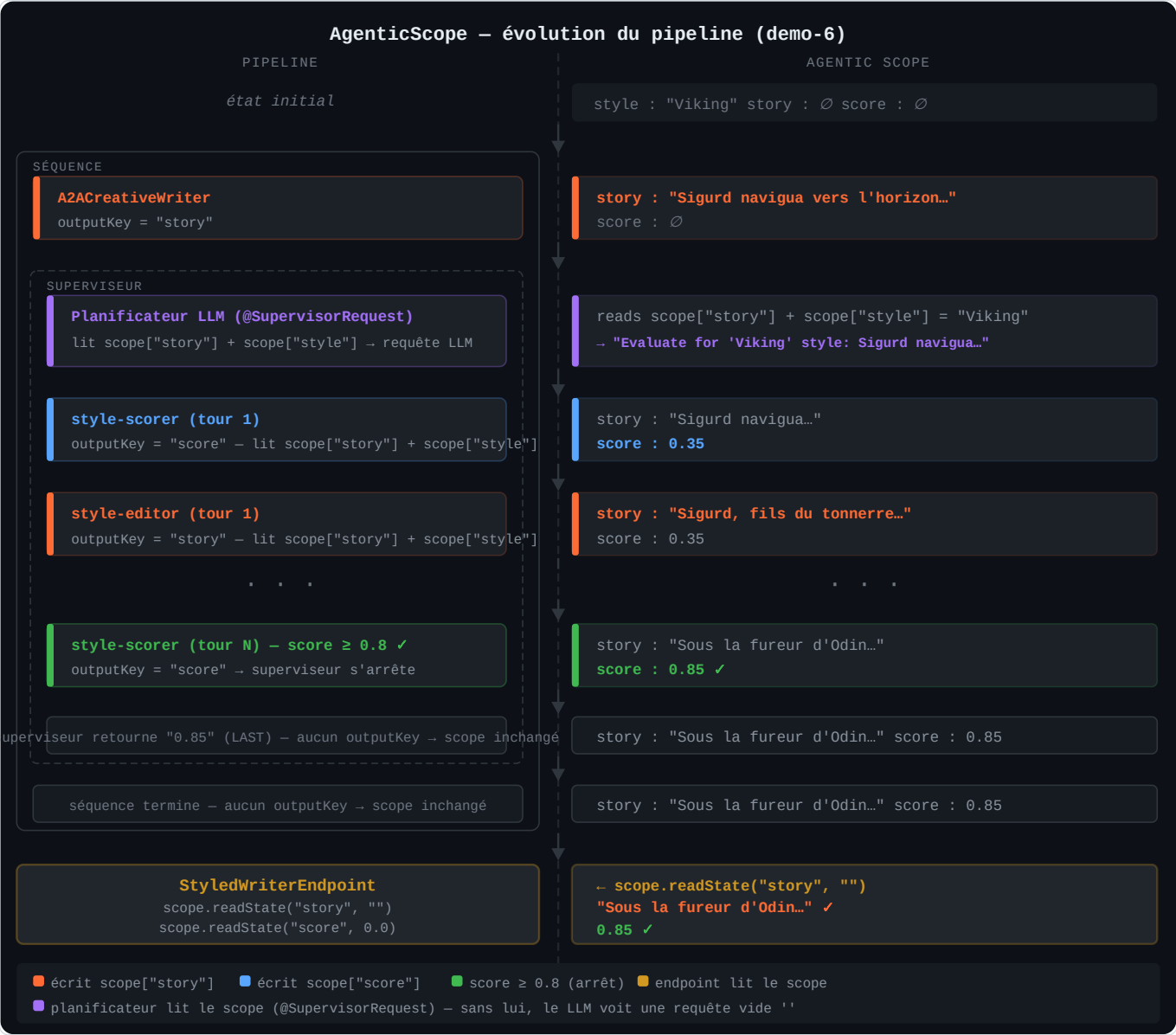
// L'endpoint lit ensuite :
// scope.readState("story", "")
```

③ Lecture par @SupervisorRequest

Le planificateur LLM du superviseur construit son message utilisateur depuis le scope grâce à une méthode statique `@SupervisorRequest`. Sans elle, le planificateur reçoit une requête vide `''` et ne peut pas prendre de décision éclairée.

```
@SupervisorRequest
static String buildRequest(
    @V("story") String story,
    @V("style") String style) {
    return "Refine for style '"
        + style + "':\n" + story;
}
// Le LLM voit :
// "The user request is: 'Refine.."
```

Le diagramme ci-dessous montre l'évolution du scope étape par étape :



⚠ PRÉREQUIS

Cet exercice réutilise le **Creative Writer** (port 8080) et le **Style Scorer** (port 8081) de l'exercice 5 — même code, mêmes ports. Seul l'orchestrateur (port 8082) change. Assurez-vous qu'Ollama tourne avec `qwen2.5:7b`.

1 Démarrer les agents serveurs (identiques à l'exercice 5)

Les services **Creative Writer** et **Style Scorer** sont inchangés par rapport à l'exercice 5. Utilisez les mêmes binaires :

```
# Terminal 1 : Creative Writer (port 8080) — identique à l'exercice 5
cd demo-project/demo-5-a2a/solution/a2a-creative-writer # ou base si déjà construit
mvn clean install
./target/server/bin/standalone.sh # Linux / macOS
target\server\bin\standalone.bat # Windows

# Terminal 2 : Style Scorer (port 8081) — identique à l'exercice 5
cd demo-project/demo-5-a2a/solution/a2a-style-scorer
mvn clean install
./target/server/bin/standalone.sh -Djboss.socket.binding.port-offset=1 # Linux / macOS
target\server\bin\standalone.bat -Djboss.socket.binding.port-offset=1 # Windows
```

💡 RÉUTILISEZ LES SERVEURS DE L'EXERCICE 5

Si vous avez déjà les serveurs de l'exercice 5 en cours d'exécution sur les ports 8080 et 8081, laissez-les tourner. Seul l'orchestrateur (port 8082) utilisera le module `demo-6-supervisor`.

2 Câbler les proxies A2A et annoter le superviseur

Étapes 1 & 2 — Proxies A2A (`A2ACreativeWriter.java` , `A2AStyleScorer.java`)

Identiques à l'exercice 5 — ajoutez simplement `@RegisterA2AAgent` sur chaque interface :

```
@RegisterA2AAgent(
    name = "creative-writer",
    a2aServerUrl = "${a2a.creative-writer.url}",
    outputKey = "story",
    description = "Generate a Norse saga based on the given topic")
public interface A2ACreativeWriter {
    String generateStory(@V("topic") String topic);
}

@RegisterA2AAgent(
    name = "style-scorer",
    a2aServerUrl = "${a2a.style-scorer.url}",
    outputKey = "score",
    description = "Score a saga based on how well it captures a given style")
public interface A2AStyleScorer {
    double scoreStyle(@V("story") String story, @V("style") String style);
}
```

Étape 3 — Éditeur local (`StyleEditor.java`)

```
@RegisterSimpleAgent(
    name = "style-editor",
    description = "Reforge a saga to better capture a given style",
    outputKey = "story",
    chatModelName = "ollama")
public interface StyleEditor {
    @UserMessage("""
        You are a master Norse skald who shapes and tempers sagas like a smith forges steel.
        Rewrite the following saga to better honor and embody the {{style}} style.
        Return only the saga and nothing else.
        The saga is "{{story}}".
        """)
    String editStory(@V("story") String story, @V("style") String style);
}
```

Étape 4 — Le superviseur LLM (`StyleReviewSupervisor.java`) ← nouveauté !

Ouvrez `StyleReviewSupervisor.java` et ajoutez l'annotation `@RegisterSupervisorAgent`. Le superviseur remplace à lui seul la boucle *et* la condition de sortie de l'exercice 5. Notez les trois éléments clés : `extends AgenticScopeAccess`, `@SupervisorRequest`, et le type de retour `ResultWithAgenticScope<String>` :

```
import dev.langchain4j.agentic.declarative.SupervisorRequest;
import dev.langchain4j.agentic.scope.AgentScopeAccess;
import dev.langchain4j.agentic.scope.ResultWithAgentScope;
import dev.langchain4j.agentic.supervisor.SupervisorResponseStrategy;
import dev.langchain4j.cdi.spi.RegisterSupervisorAgent;
import dev.langchain4j.service.V;

@RegisterSupervisorAgent(
    name = "style-review-supervisor",
    subAgentNames = {"style-scorer", "style-editor"},
    chatModelName = "ollama",
    maxAgentsInvocations = 10,
    supervisorContext = """
        You are a Viking saga quality judge overseeing a story refinement process.
        Use the style-scorer to evaluate how well the story matches the requested style (score 0.0-1.0).
        Use the style-editor to rewrite the story if the score is below 0.8.
        Repeat score → edit cycles until the score reaches 0.8 or you have made 5 refinements.
        """,
    supervisorResponseStrategy = SupervisorResponseStrategy.LAST)
public interface StyleReviewSupervisor extends AgentScopeAccess { // (1)

    @SupervisorRequest // (2)
    static String buildRequest(@V("story") String story,
                              @V("style") String style) {
        return "Evaluate and if necessary refine the following story"
            + " to better match the '" + style + "' style:\n\n" + story;
    }

    ResultWithAgentScope<String> refineStory( // (3)
        @V("story") String story, @V("style") String style);
}
```

⚠ PIÈGE CRITIQUE : SANS `@SUPERVISORREQUEST`, LE PLANIFICATEUR VOIT UNE REQUÊTE VIDE

Sans méthode statique `@SupervisorRequest` (2), le planificateur LLM reçoit `"The user request is: ''"`. Il voit les scores arriver dans le scope mais ne sait pas quelle story évaluer ni quel style cibler. Il peut alors déclarer le travail terminé dès le premier score, retournant une story vide.

`@SupervisorRequest` lit les clés du scope (via les paramètres `@V`) et retourne la chaîne que le LLM recevra comme requête utilisateur. C'est l'équivalent déclaratif de `.requestGenerator(scope → ...)` de l'API impérative.

⚠ PIÈGE : N'AJOUTEZ PAS `OUTPUTKEY` SUR LE SUPERVISEUR !

Avec `supervisorResponseStrategy = LAST`, le superviseur retourne la réponse du *dernier sous-agent appelé*. Si le superviseur termine par `style-scorer` (pour vérifier la qualité), `LAST` retourne le **score** — pas la story. Si vous ajoutiez `outputKey = "story"`, ce score écraserait le vrai contenu dans `scope["story"]`.

Solution : **omettre** `outputKey` sur le superviseur. C'est `style-editor` qui écrit dans `scope["story"]` via son propre `outputKey = "story"`. Le score reste accessible via `scope.readState("score")`.

💡 `SUPERVISORCONTEXT` VS `EXITCONDITIONNAME`

Dans l'exercice 5, la condition de sortie était un bean CDI Java (`Predicate<AgentScope>`) qui testait `score ≥ 0.8`. Dans cet exercice, `supervisorContext` est un prompt texte qui *explique* au LLM quand s'arrêter. Le LLM interprète les résultats et décide dynamiquement — plus flexible, mais moins déterministe.

3 Pipeline, service et endpoint

Étape 5 — Déclarer le pipeline (`StyledWriter.java`)

Similaire à l'exercice 5, mais avec `style-review-supervisor` à la place de `style-review-loop` et **sans** `outputKey` sur la séquence (voir encart ci-dessous) :

```
import dev.langchain4j.agentic.scope.AgentScopeAccess;
import dev.langchain4j.agentic.scope.ResultWithAgentScope;
import dev.langchain4j.cdi.spi.RegisterSequenceAgent;
import dev.langchain4j.service.V;

@RegisterSequenceAgent(
    name = "styled-writer",
    subAgentNames = {"creative-writer", "style-review-supervisor"})
public interface StyledWriter extends AgentScopeAccess {
    ResultWithAgentScope<String> writeStoryWithStyle(
        @V("topic") String topic, @V("style") String style);
}
```

⚠ PAS D' `OUTPUTKEY` SUR LA SÉQUENCE

Dans l'exercice 5, `@RegisterSequenceAgent(outputKey = "story")` fonctionnait parce que la boucle retournait toujours la story. Ici, le superviseur avec `LAST` peut retourner le *score* (si son dernier appel était `style-scorer`). Sans `outputKey` sur la séquence, `scope["story"]` conserve la dernière story écrite par `style-editor` via son propre `outputKey`. On lit ensuite la story directement depuis le scope (étape 7).

Étape 6 — Simplifier `OrchestratorService.java`

Remplacez tout le câblage impératif `@PostConstruct` par un simple `@Inject` et mettez à jour la méthode pour retourner le `ResultWithAgentScope` :

```
import dev.langchain4j.agentic.scope.ResultWithAgentScope;
import jakarta.enterprise.context.ApplicationScoped;
import jakarta.inject.Inject;

@ApplicationScoped
public class OrchestratorService {

    @Inject
    StyledWriter styledWriter;

    public ResultWithAgentScope<String> writeStyledStory(String topic, String style) {
        return styledWriter.writeStoryWithStyle(topic, style);
    }
}
```

Étape 7 — Exposer story et score (`StyledWriterEndpoint.java`)

Mettez à jour l'endpoint pour lire story et score directement depuis le scope — les deux viennent des sous-agents qui y ont écrit via leur propre `outputKey` :

```
import dev.langchain4j.agentic.scope.AgentScope;
import dev.langchain4j.agentic.scope.ResultWithAgentScope;

// Dans writeStyledStory() :
ResultWithAgentScope<String> result = orchestratorService.writeStyledStory(topic, style);
AgentScope scope = result.agentScope();
String story = scope.readState("story", ""); // écrit par style-editor (outputKey="story")
double score = scope.readState("score", 0.0); // écrit par style-scorer (outputKey="score")

String json = ""
    {
        "story": %s,
        "score": %s,
        "topic": %s,
        "style": %s
    }
    """.formatted(jsonString(story), score, jsonString(topic), jsonString(style));
```

💡 POURQUOI PAS `RESULT.RESULT()` ?

Sans `outputKey` sur la séquence, `result.result()` renvoie la valeur brute du dernier sous-agent (le superviseur). Avec `LAST`, c'est souvent le score du dernier appel au `style-scorer`. Lire depuis le scope est fiable car `style-editor` y écrit toujours la story via son `outputKey = "story"`.

Démarrer l'orchestrateur

```
# Terminal 3 : Orchestrateur Supervisor (port 8082)
cd demo-project/demo-6-supervisor/base/a2a-orchestrator
mvn clean install
./target/server/bin/standalone.sh -Djboss.socket.binding.port-offset=2 # Linux / macOS
target\server\bin\standalone.bat -Djboss.socket.binding.port-offset=2 # Windows
```

⚠ ORDRE DE DÉMARRAGE

Démarrez le Creative Writer (8080) et le Style Scorer (8081) **avant** l'Orchestrateur Supervisor (8082).

✓

Tester et valider l'exercice 6

Via curl

```
# Forger une saga avec un superviseur LLM
curl "http://localhost:8082/api/styled-story?topic=Erik+le+Rouge+traverse+les+mers&style=epique"

# La réponse inclut la story ET le score final
# { "story": "...", "score": 0.85, "topic": "...", "style": "..." }
```

Via l'interface web

Ouvrez <http://localhost:8082> (<http://localhost:8082>) pour accéder à l'interface Story Forge. Comparez les logs avec l'exercice 5 : la boucle est pilotée par le LLM, pas par un Predicate Java.

- ✓ Les trois services WildFly démarrent sans erreur (ports 8080, 8081, 8082)
- ✓ La réponse JSON contient à la fois `story` et `score`
- ✓ La `story` est bien une saga, pas un score numérique
- ✓ Les logs montrent le superviseur appelant scorer et éditeur alternativement
- ✓ Le score final affiché est ≥ 0.8 (ou le superviseur a atteint la limite d'invocations)

💡 BLOQUÉ ?

Passez à la solution : utilisez les modules sous `demo-6-supervisor/solution/` au lieu de `base/`.

🎉 FÉLICITATIONS — ATELIER TERMINÉ !

Vous avez construit six applications alimentées par l'IA avec LangChain4j-CDI : un skald viking injectable, un guide résilient avec observabilité, un jeu de dés MCP, un skald avec guardrails, un pipeline A2A avec boucle déclarative, et maintenant un orchestrateur piloté par le LLM. Tout ça avec les patterns Jakarta EE que vous connaissiez déjà.

Et après ?

Modules LangChain4j-CDI

MODULE	ARTEFACT	UTILITÉ
core	langchain4j-cdi-core	API + annotations
portable	langchain4j-cdi-portable-ext	WildFly, Payara, GlassFish, Liberty
build	langchain4j-cdi-build-compatible-ext	Quarkus, Helidon
config	langchain4j-cdi-config	MicroProfile Config SPI
FT	langchain4j-cdi-fault-tolerance	Retry, Timeout, CircuitBreaker
telemetry	langchain4j-cdi-telemetry	Spans et métriques OpenTelemetry

Ressources

- [LangChain4j-CDI on GitHub](https://github.com/langchain4j/langchain4j-cdi) (<https://github.com/langchain4j/langchain4j-cdi>)
- [LangChain4j Documentation](https://docs.langchain4j.dev/) (<https://docs.langchain4j.dev/>)
- [MCP Specification](https://spec.modelcontextprotocol.io/) (<https://spec.modelcontextprotocol.io/>)
- [WildFly Documentation](https://docs.wildfly.org/) (<https://docs.wildfly.org/>)

Dépannage

PROBLÈME	CAUSE	SOLUTION
<code>ClassNotFoundException: ... faultTolerance.Retry</code>	Couches Galleon manquantes	Ajouter <code>microprofile-fault-tolerance</code> dans <code><layers></code>
Propriétés ignorées silencieusement	Préfixe <code>.config.</code> manquant	Utiliser <code>...plugin.X.config.prop=val</code>
La mémoire se réinitialise à chaque message	<code>get()</code> crée une nouvelle instance	Utiliser <code>computeIfAbsent()</code> dans le provider
<code>ChatMessage.text()</code> ne compile pas	<code>ChatMessage</code> n'a pas de <code>text()</code>	Pattern-match : <code>SystemMessage</code> , <code>UserMessage</code> , etc.
404 sur la racine du contexte	Pas de <code><name></code> dans le plugin wildfly	Ajouter <code><name>demo-N</name></code>
Le serveur MCP ne répond pas	Le serveur ne tourne pas sur le port 8090	Démarrer le serveur MCP d'abord
<code>MISTRAL_API_KEY</code> introuvable	Variable d'environnement non définie	<code>export MISTRAL_API_KEY=your-key</code>
Guardrail jamais déclenché	<code>@Named</code> manquant ou nom différent	Vérifier que le nom dans <code>@Named("...")</code> correspond exactement à <code>inputGuardrailNames</code>
<code>NullPointerException</code> dans le guardrail	Détecteur non initialisé	Vérifier que <code>@PostConstruct</code> appelle bien <code>new OptimizeLangDetector().loadModels()</code>
Texte français rejeté par <code>french-input</code>	Texte trop court pour Tika	Normal — la logique <code>isReasonablyCertain()</code> donne le benefit of the doubt aux textes ambigus
Réponse 400 sans message lisible	Exception guardrail non parsée	Le <code>ChatResource</code> extrait le message avec <code>extractGuardrailMessage()</code> — vérifier l'implémentation
L'orchestrateur ne démarre pas	Agents A2A non accessibles	Démarrer Creative Writer (8080) et Style Scorer (8081) avant l'Orchestrateur (8082)
<code>Connection refused</code> vers l'agent A2A	Port-offset manquant	Utiliser <code>-Djboss.socket.binding.port-offset=1</code> pour le scorer, <code>=2</code> pour l'orchestrateur
Score toujours < 0.8 après 5 itérations	Modèle trop petit ou style vague	Normal — la boucle s'arrête après <code>maxIterations(5)</code> . Essayez un style plus simple.
<code>AgentCard</code> non trouvée	<code>@PublicAgentCard</code> manquant	Vérifier que le producteur CDI est annoté <code>@Produces @PublicAgentCard</code>
La réponse JSON contient un score au lieu d'une story	<code>outputKey = "story"</code> sur <code>@RegisterSupervisorAgent</code> ou sur <code>@RegisterSequenceAgent</code> avec stratégie <code>LAST</code>	Supprimer <code>outputKey</code> du superviseur ET de la séquence. Lire la story via <code>scope.readState("story", "")</code> dans l'endpoint.
Le superviseur ne s'arrête pas ou dépasse <code>maxAgentsInvocations</code>	<code>supervisorContext</code> ambigu ou modèle trop petit	Formuler le critère d'arrêt clairement (seuil numérique explicite). Essayez <code>qwen2.5:7b</code> ou un modèle plus grand.
<code>style-review-supervisor</code> non trouvé au démarrage	<code>@RegisterSupervisorAgent</code> manquant ou <code>name</code> incorrect	Vérifier que <code>name = "style-review-supervisor"</code> correspond exactement à <code>subAgentNames</code> dans <code>@RegisterSequenceAgent</code>

⚠ SECOURS D'URGENCE

Si quelque chose ne va pas, passez au module solution :

```
# Exemple pour l'exercice 4 :
cd demo-4-guardrails/solution && mvn clean install
./target/server/bin/standalone.sh          # Linux / macOS
target\server\bin\standalone.bat           # Windows

# Exemple pour l'exercice 5 (3 terminaux) :
cd demo-5-a2a/solution/a2a-creative-writer && mvn clean install
./target/server/bin/standalone.sh          # Linux / macOS (port 8080)

cd demo-5-a2a/solution/a2a-style-scorer && mvn clean install
./target/server/bin/standalone.sh -Djboss.socket.binding.port-offset=1    # port 8081

cd demo-5-a2a/solution/a2a-orchestrator && mvn clean install
./target/server/bin/standalone.sh -Djboss.socket.binding.port-offset=2    # port 8082

# Exemple pour l'exercice 6 (orchestrateur seulement – réutiliser les serveurs de l'ex5) :
cd demo-6-supervisor/solution/a2a-orchestrator && mvn clean install
./target/server/bin/standalone.sh -Djboss.socket.binding.port-offset=2    # port 8082
```